



0.1 Objetivo

O objetivo deste exercício é consolidar os conceitos de **modularização** (separação entre interface e implementação) , **automação de compilação** via Makefile e **sobrecarga de operadores** para criar tipos de dados intuitivos em C++.

0.2 Contexto

Em motores de física e jogos, vetores bidimensionais são fundamentais. Você deve implementar uma biblioteca modular para manipular a classe **Vetor2D**, permitindo operações matemáticas com a mesma sintaxe de tipos primitivos.

0.3 Requisitos do Código

O projeto deve ser obrigatoriamente dividido em três arquivos:

- **Vetor2D.h**: Deve conter a definição da classe, atributos privados x e y (float) e os protótipos dos operadores. É obrigatório o uso de *header guards* (`#ifndef`, `#define`, `#endif`).
- **Vetor2D.cpp**: Deve conter a implementação real de cada operador declarado no cabeçalho.
- **main.cpp**: Deve conter a função principal que testa a biblioteca.

0.3.1 Operadores a Implementar

Implemente as seguintes sobrecargas:

1. **Soma (+)**: $(x_1, y_1) + (x_2, y_2) = (x_1 + x_2, y_1 + y_2)$.
2. **Igualdade (==)**: Retorna verdadeiro se as coordenadas forem idênticas.
3. **Saída (<<)**: (Função `friend`) Permite imprimir o vetor no formato (x, y) diretamente com `std::cout`.

0.4 Requisitos do Makefile

Você deve criar um arquivo **Makefile** para gerenciar o projeto seguindo estas regras:

- Definir variáveis para o compilador (`CC = g++`) e flags (`CFLAGS = -Wall -O2`).
- Criar regras separadas para gerar os arquivos objetos **Vetor2D.o** e **main.o**.
- O alvo principal (*target*) deve gerar um executável chamado **PhysicsEngine**.
- Incluir um alvo **clean** para limpeza de binários.

0.5 Exemplo de Uso Esperado

```
int main() {
    Vetor2D v1(10.5, 20.0);
    Vetor2D v2(5.0, 10.0);
    Vetor2D v3 = v1 + v2;

    std::cout << "Resultado: " << v3 << std::endl;

    if (v1 == v2)
        std::cout << "Iguais" << std::endl;

    return 0;
}
```